

# Servlet & JSP

Ms.Shweta Rajput

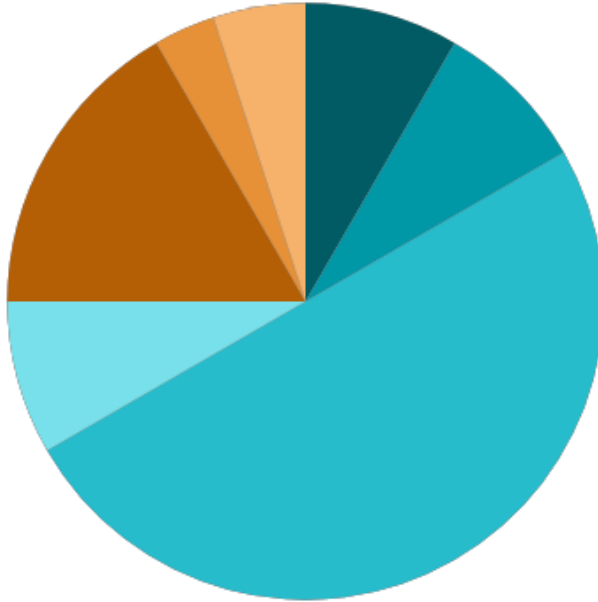
Department of Computer Science & Engineering



# Outline

1. J2EE Architecture
2. Enterprise Application Concepts, N-Tier Application Concepts
3. Servlet Model: Overview of Servlet, Servlet Life Cycle
4. HTTP Methods, Structure, Deployment Descriptor
5. RequestDispatcher Interface
6. The Filter API: Filter and FilterChain
7. Cookies and Session Management: State and Session Handling, Session Timeout and Session Tracking
8. JSP Directives, JSP Implicit Objects, JSP Form Processing
9. JSP Session and Cookies Handling, JSP Database Access, JSP Exception Handling

# Lecture Structure



|  |            |                                               |
|--|------------|-----------------------------------------------|
|  | 05 Minutes | Outline & Objectives                          |
|  | 05 Minutes | Recap of Previous Lecture                     |
|  | 30 Minutes | Delivery of Actual Content as per Lesson Plan |
|  | 05 Minutes | Industry Relevance                            |
|  | 10 Minutes | Interactive Activities                        |
|  | 02 Minutes | Importance of Discipline                      |
|  | 03 Minutes | Recording of Attendance                       |

Reference: Office Order No. Provost/092023135, Date: 18<sup>th</sup> September 2023



# Objectives

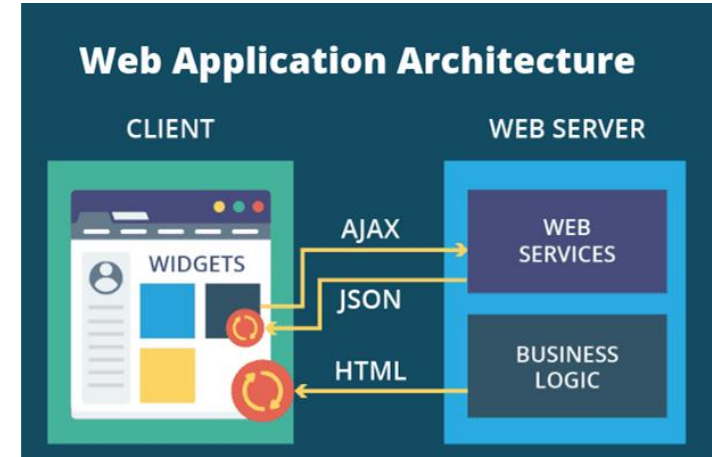
1. To understand the J2EE architecture and core enterprise application concepts.
2. To explain N-Tier application architecture and its role in scalable systems.
3. To study the Servlet model, life cycle, and HTTP request–response handling.
4. To implement request flow control using RequestDispatcher and Filters.
5. To manage user state using cookies and session management techniques.
6. To develop dynamic web applications using JSP directives, implicit objects, and form processing.
7. To handle database connectivity and exception handling in JSP-based applications.

# Web Applications

- Need for applications accessible anytime, anywhere
- Platform-independent and scalable solutions
- Real-time data processing and interaction
- Backbone of modern enterprise systems

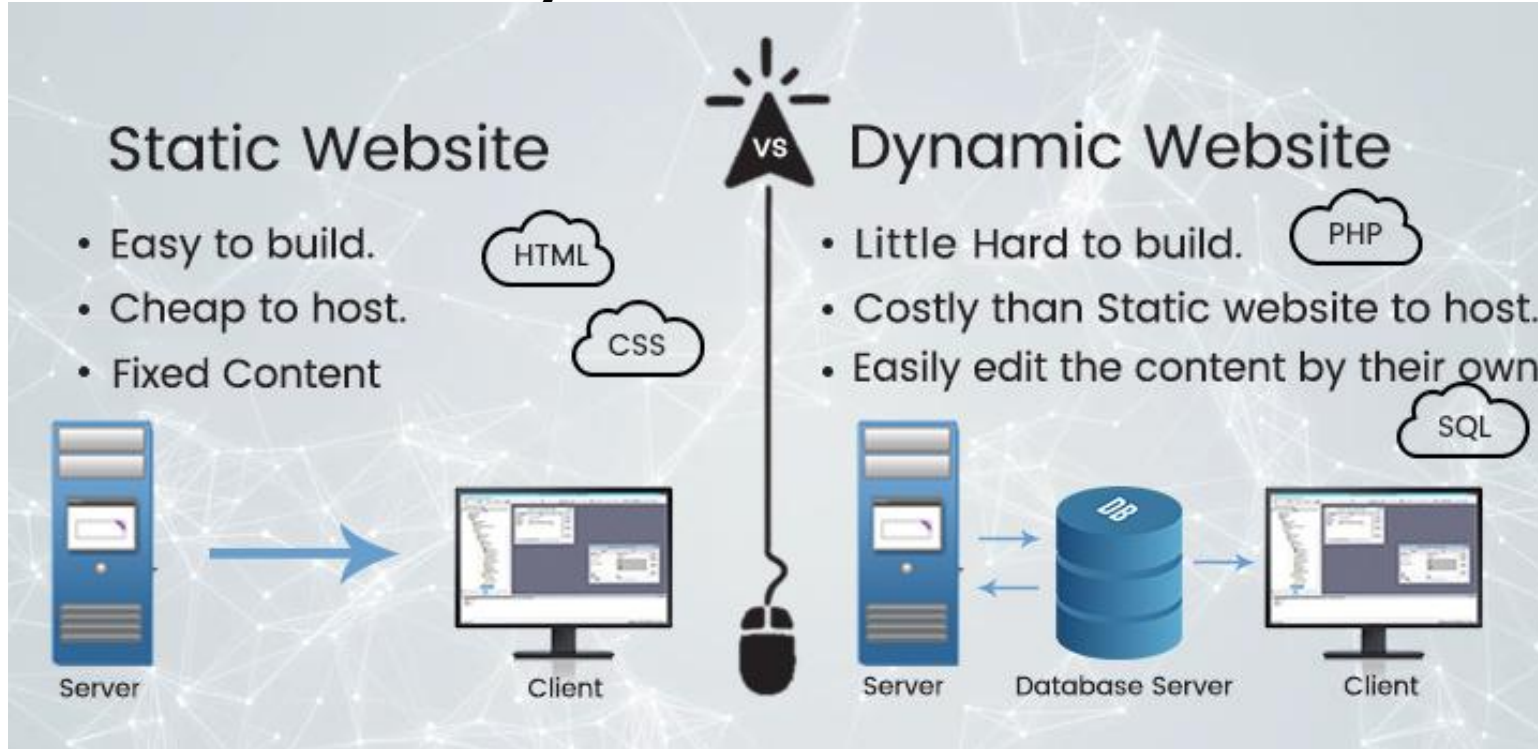
## Website

- Collection of related web pages
- Accessible using a web browser
- Hosted on a web server
- Identified by a domain name or IP address



<https://networkencyclopedia.com/web-application/>

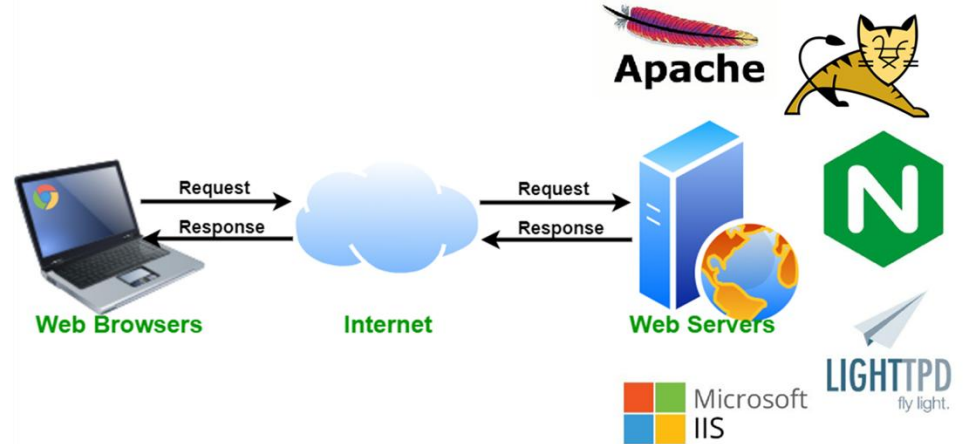
# Static Website Vs Dynamic Website



<https://networkencyclopedia.com/web-application/>

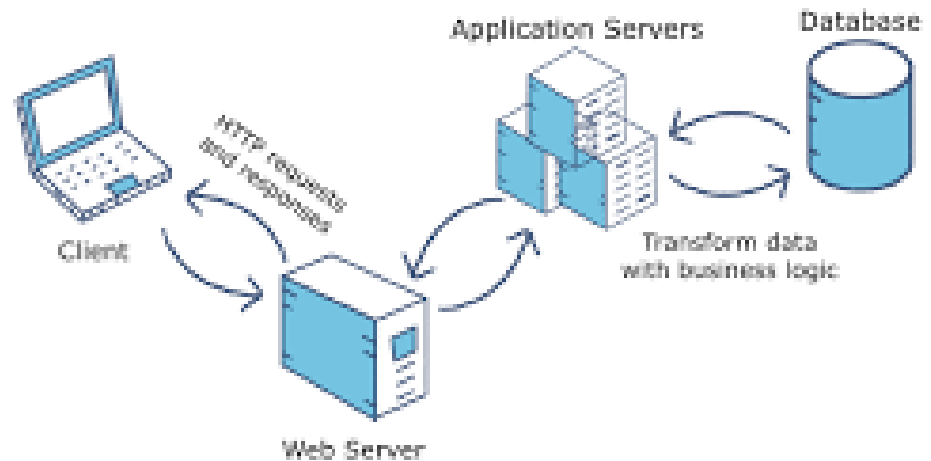
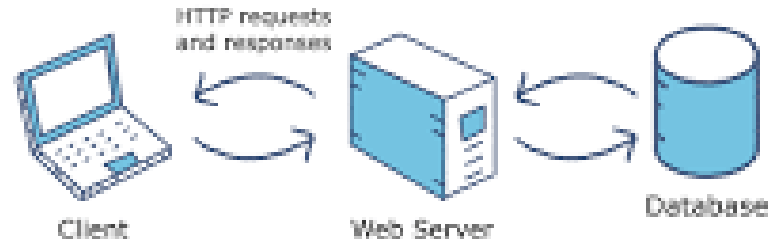
# Web Server

- Handles HTTP requests from clients
- Serves static content (HTML, CSS, images)
- Examples:
  - Apache HTTP Server
  - Nginx
  - Listens on port 80 / 443.



# Application Server

- Executes business logic
- Handles dynamic content
- Supports server-side technologies
- Examples:
  - Apache Tomcat
  - JBoss
  - GlassFish



<https://www.educative.io/answers/web-server-vs-application-server>





# WebContainer

- In Jakarta EE, the web container (also known as a servlet container) is a core component of the application server that manages the execution and lifecycle of web components such as [Jakarta Servlets](#), [Jakarta Server Pages \(JSP\)](#), and [Jakarta Faces](#).
- It provides a runtime environment that adds a layer of services between the application and the underlying server.



# Enterprise Architecture

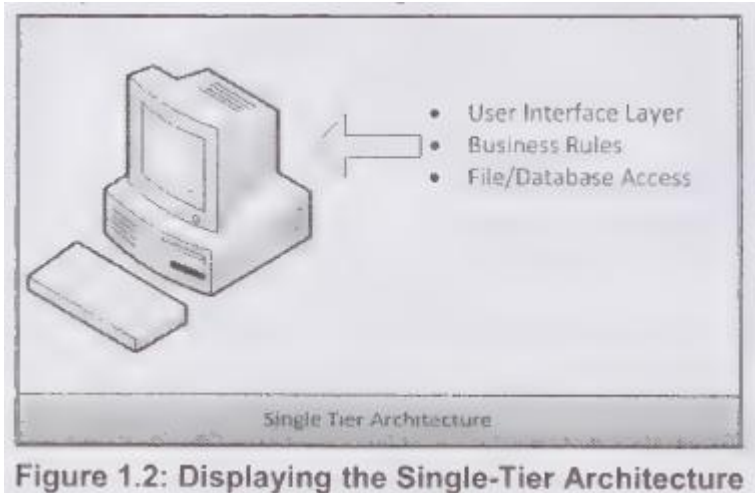
An enterprise application is divided into three logical layers:

1. Presentation Layer: User interface that displays data, collects user input, and validates actions.
2. Business Logic Layer: Implements application rules and processes business logic.
3. Data Access Layer: Manages data storage, retrieval, and database interactions.

Types of enterprise architectures:

1. The single-tier architecture
2. The 2-tier architecture
3. The 3-tier architecture
4. The n-tier architecture

# The single-tier architecture



- A single-tier architecture consists of the presentation logic, the business rules, and the data access layers—in a single computing architecture.
- Applications created on the single-tier architecture are relatively easy to manage and implement data consistency, as data is stored at a single location.
- The only problem is that such applications cannot be scaled up to handle multiple users, and they do not provide an easy means of sharing data across an organization.

# The two-tier architecture

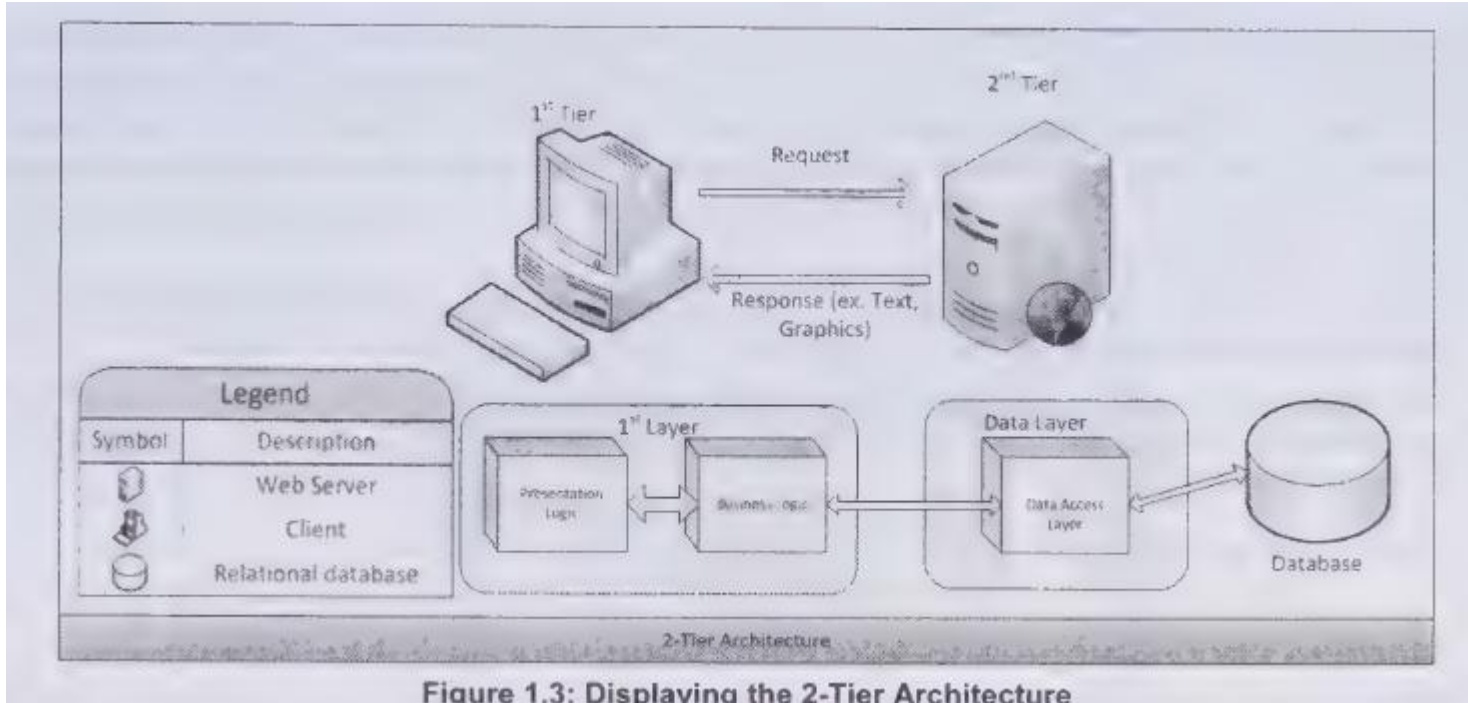


Figure 1.3: Displaying the 2-Tier Architecture

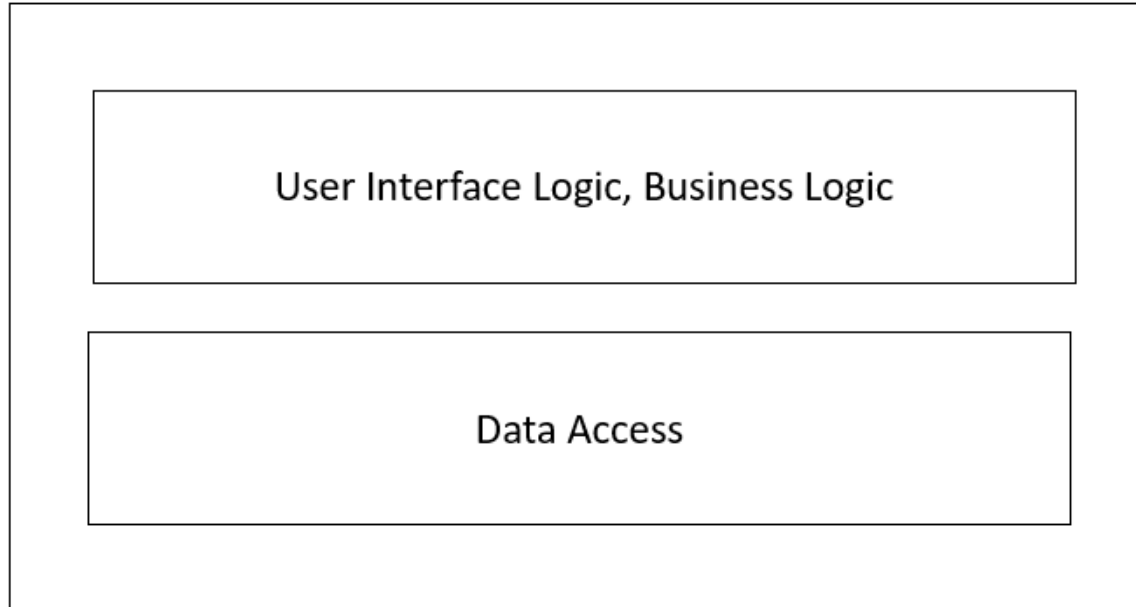


## The two-tier architecture

- The 2-tier architecture separates the data access layer and the business logic layer. This type of architecture is generally data driven; with the application existing entirely on the local machine and the database deployed at a specific and secure location in an organization.
- This type of architecture is the traditional method of enterprise development.
- In a 2-tier application, the processing load is entrusted to the client, while the server simply controls the traffic between the application and the data access layers.
- The main objective of the 2-tier architecture is to centralize the data so that multiple users can access a common database at a given time.
- Due to centralization of data, the load of executing an application is shared with the central database server.



# The two-tier architecture





## The two-tier architecture

- When a change occurs in a database, some users may not be ready to implement the changes, while others may insist on making the changes effective immediately. This may result in different client installations by using different versions of applications, resulting in inconsistency of the database. You may think that this problem can be solved by developing a reusable library encapsulating the business rules.
- In other words, a change in any rule can be simply replaced in that library and then the application can be rebuilt and redistributed. However, there are few inherent problems in this regard:
  - You need to assume that all applications are created by using the same programming language
  - You need to run all applications on the same platform, leading to heavy load on a system
  - You need to recompile or reassemble the application along with the changes implemented in a new library



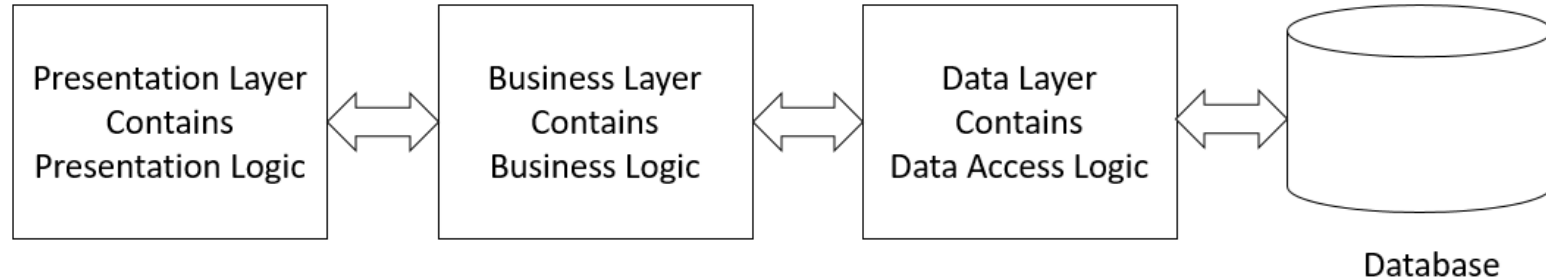
# The three-tier architecture

In the 3-tier architecture, an application is virtually split into three separate logical layers. The 3-tier architecture has the following layers:

- **First tier** - Refers to the presentation layer, which consists of a Graphical User Interface (GUI) to interact with a user.
- **Middle tier** - Refers to the business layer, which consists of the business logic for an application. The middle tier represents the code that is called by a user through the presentation layer to retrieve data from the data layer.
- **Third tier** - Refers to the data layer, which contains the data access logic needed for the application.



# The three-tier architecture



- As the business logic and the user interface are at different layers, it adds a lot of flexibility when designing an application in the 3-tier architecture, as compared to the 2-tier architecture.
- By using the 3-tier architecture, multiple user interfaces can be built and deployed without changing the application logic, provided this architecture defines a clear interface to the presentation layer.



# The n-tier architecture

- In the n-tier architecture, the business logic is retrieved from the presentation layer.
- For example, a desktop application serves as a presentation layer representing the user interface, which can communicate with the business logic tier. It is no longer responsible for enforcing business rules or accessing databases.
- The n-tier architecture supports various types of configurations to define the application layer. The n-tier architecture can be virtually distributed into the following components:
  - **The user interface** - Refers to the interface between a user and a database. The interface may be a Web browser running through a firewall, a heavy desktop application, or even a wireless device.

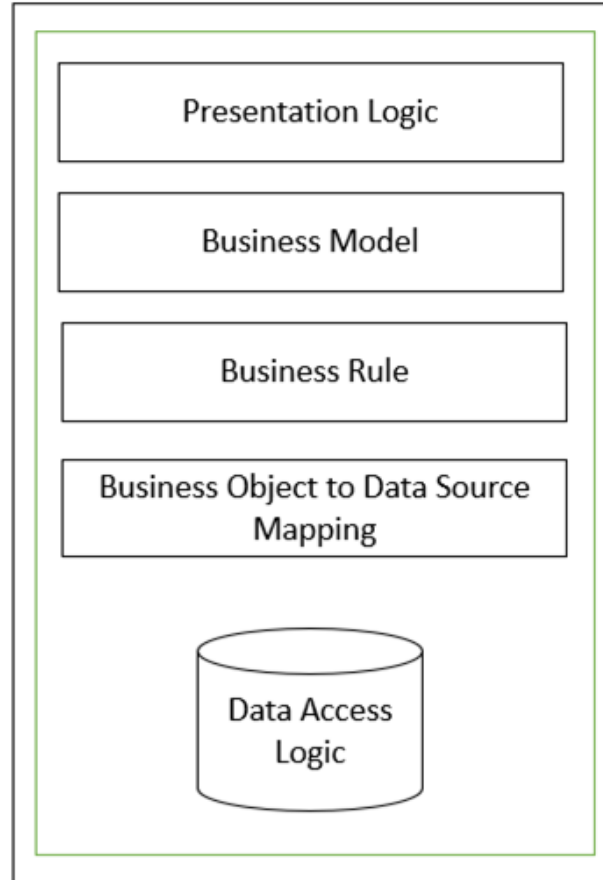


# The n-tier architecture

- **The presentation logic** - Defines what will be displayed on the user interface and how a user's requests would be handled. The business logic-Models an application's business logic or rules, often by interacting with the application data.
- **Infrastructure services** - Provide additional functionalities, such as messaging and transactional support required by an application.
- **The data layer** - Stores the data of an organization.



# The n-tier architecture





# The n-tier architecture

The n-tier architecture has the following advantages:

- Improved maintainability - Allows you to create applications that are easy to maintain
- Consistency - Allows you to develop enterprise applications consistent in terms of component designing and their association with the layer in the architecture where they are providing functionality
- Interoperability - Allows you to develop highly interoperable applications, which means that you can implement components in different layers to support different technologies and platforms
- Flexibility - Allows you to develop and design various types of components for different layers with flexibility

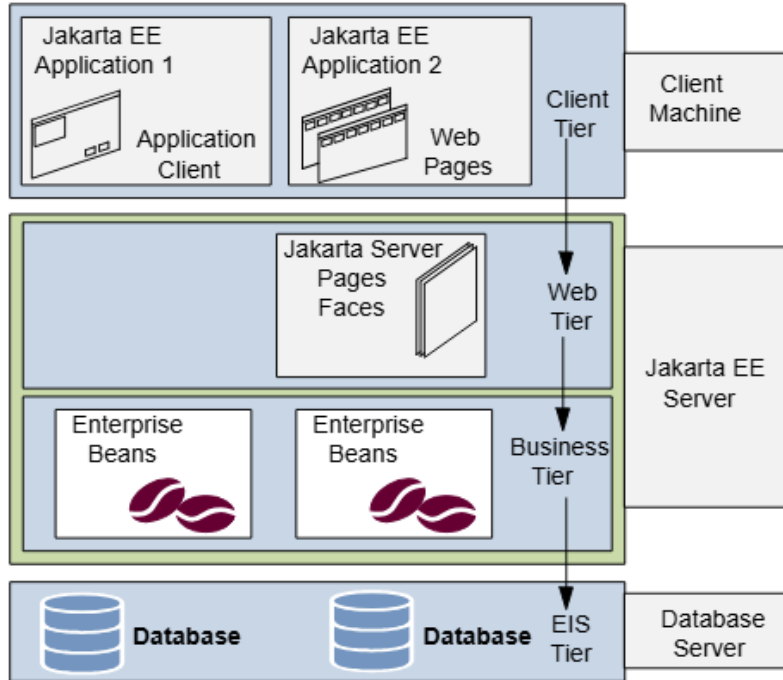


# The n-tier architecture

- Scalability - Allows you to add new components without affecting the performance of an existing application, which in turn make applications more scalable



# Jakarta EE architecture



<https://jakarta.ee/learn/docs/jakartaee-tutorial/9.1/intro/overview/overview.html>



## J2EE architecture

- The Jakarta EE platform uses a distributed multitiered application model for enterprise applications. Application logic is divided into components according to function, and the application components that make up a Jakarta EE application are installed on various machines depending on the tier in the multitiered Jakarta EE environment to which the application component belongs.
  - Client-tier components run on the client machine.
  - Web-tier components run on the Jakarta EE server.
  - Business-tier components run on the Jakarta EE server.
  - Enterprise information system (EIS)-tier software runs on the EIS server.





## J2EE architecture

- Although a Jakarta EE application can consist of all tiers shown in Figure, “Multitiered Applications”, Jakarta EE multitiered applications are generally considered to be three-tiered applications because they are distributed over three locations: client machines, the Jakarta EE server machine, and the database or legacy machines at the back end.



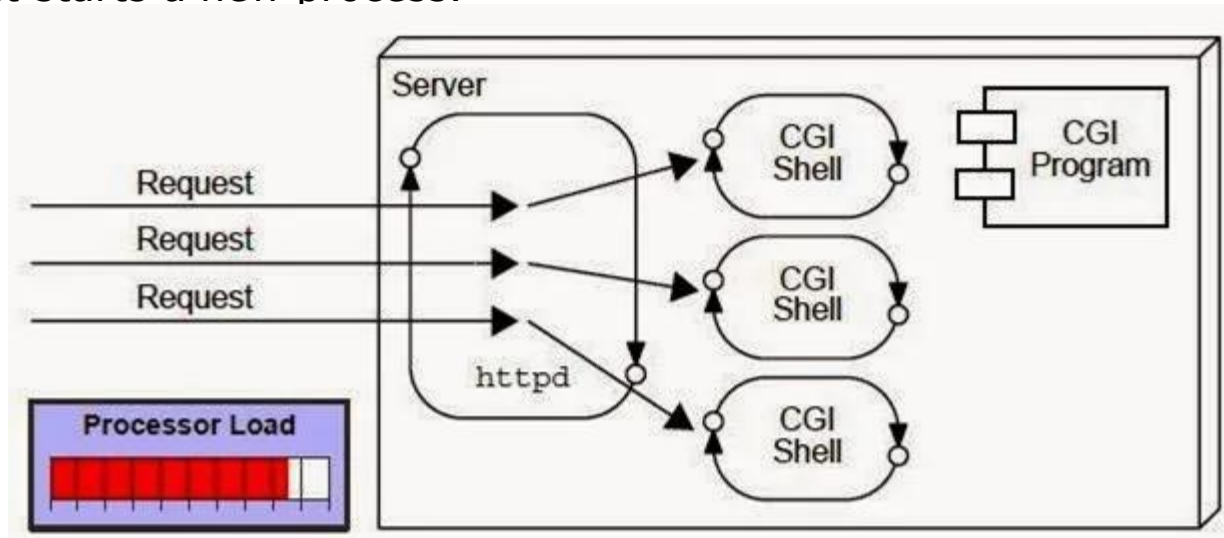
# Static Vs Dynamic Web Page

| Feature    | Static Web Page                               | Dynamic Web Page                                                 |
|------------|-----------------------------------------------|------------------------------------------------------------------|
| Content    | Fixed; same for all users                     | Varies based on user, time, or other factors                     |
| Generation | Pre-built files delivered as-is by the server | Generated in real-time on the server using scripts and databases |



## CGI(Common Gateway Interface)

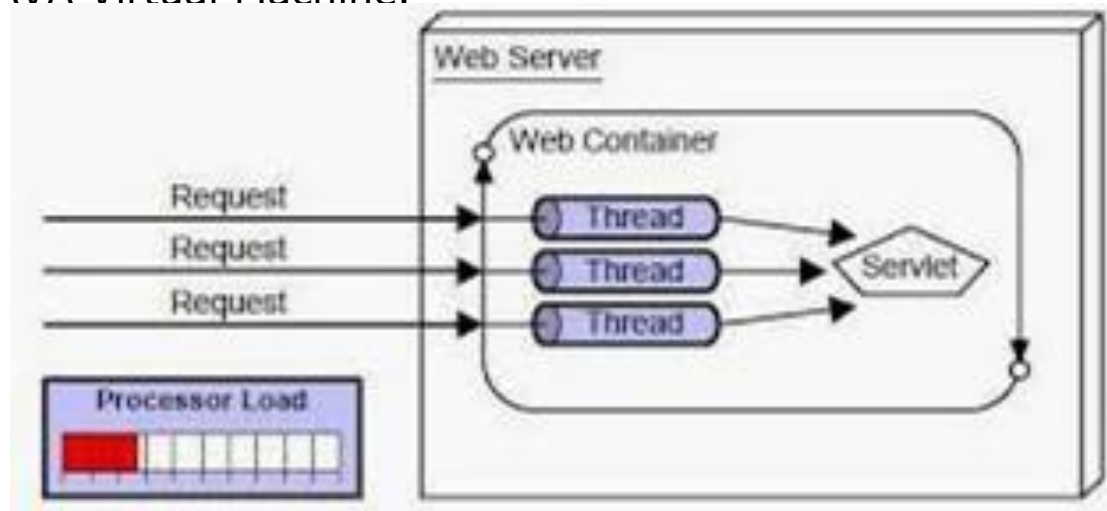
CGI technology enables the web server to call an external program and pass HTTP request information to the external program to process the request. For each request, it starts a new process.





# Java Servlets

Servlets are server based java application that can link directly to the Web server. Servlets are thread based applications and work on server level and they share data among each other. All the programs of Servlets are written in JAVA and they get to run on JAVA Virtual Machine.



# Difference Between Java Servlets and CGI

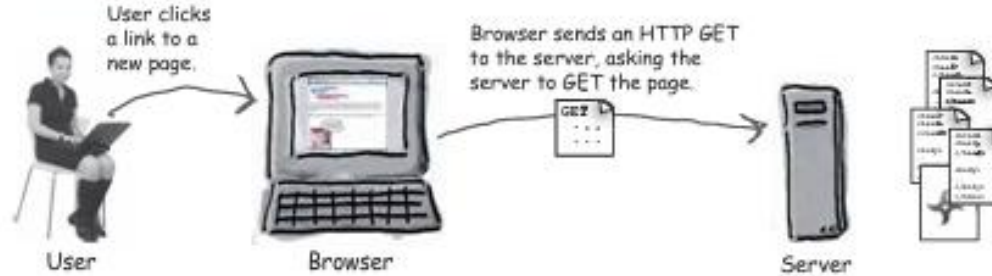
| Basis                  | CGI (Common Gateway Interface)                                                          | Servlet                                                                       |
|------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Execution Model</b> | Creates a new process for every client request.                                         | Uses a new thread for each request within a single process.                   |
| <b>Performance</b>     | Slower and less efficient due to the high overhead of process creation and destruction. | Significantly faster, as threads are more lightweight and share memory space. |
| <b>Resource Use</b>    | High memory consumption, as each process has its own address space.                     | Efficient, as all threads share the same JVM memory space.                    |
| <b>Language</b>        | Can be written in any programming language (e.g., Perl, Python, C++).                   | Written exclusively in Java, leveraging all of its features and APIs.         |



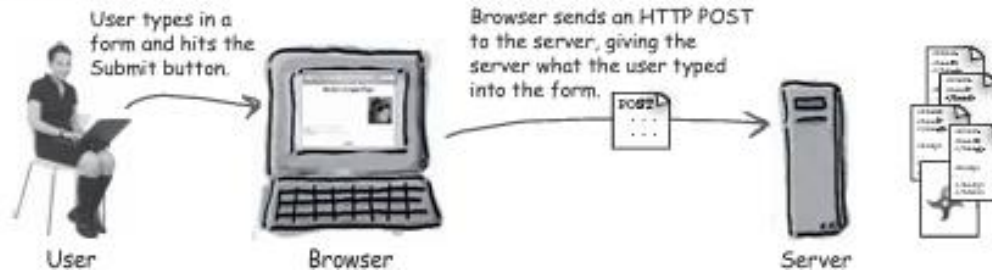
# Http Request & Response

# GET vs Post Request

## GET



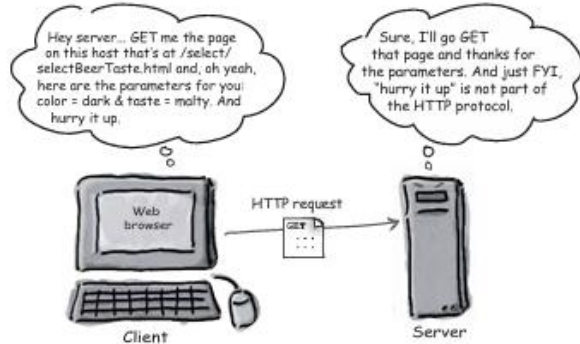
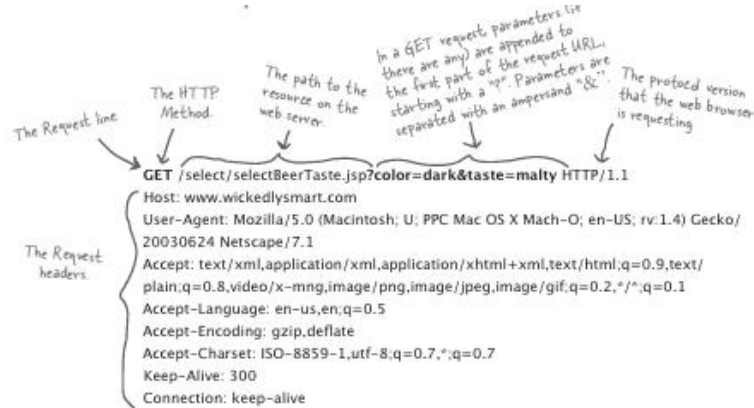
## POST



## GET

## Vs

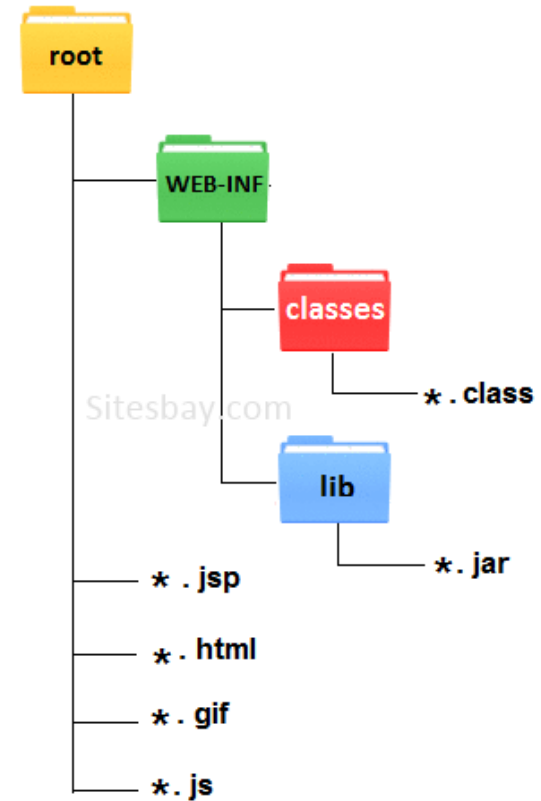
## Post Request







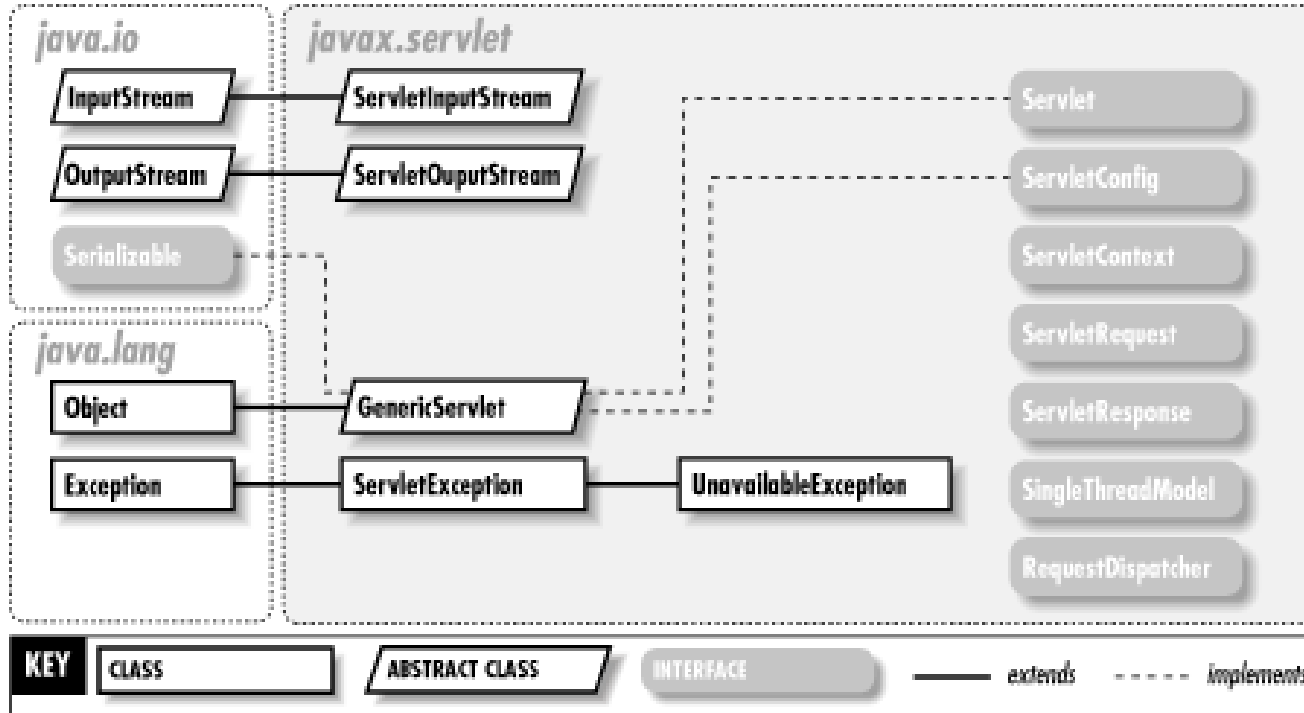
# Directory Structure



<https://www.sitesbay.com/servlet/servlet-directory-structure>



# jakarta.servlet

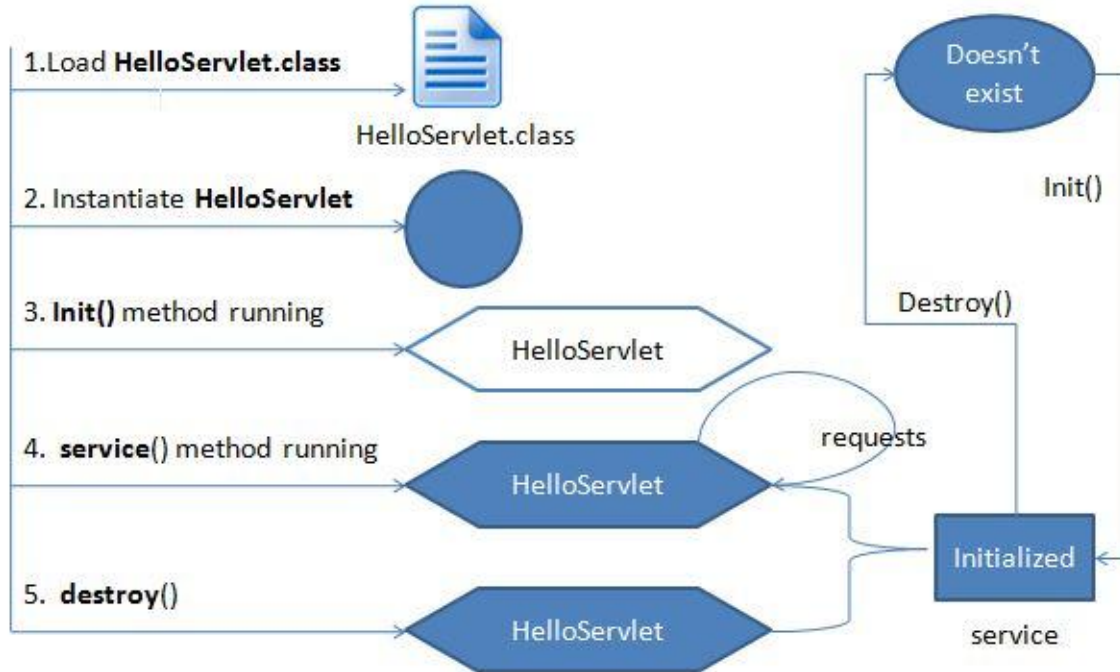


# jakarta.servlet.Servlet

| Modifier and Type    | Method                                                        | Description                                                                                                         |
|----------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| void                 | <code>destroy()</code>                                        | Called by the servlet container to indicate to a servlet that the servlet is being taken out of service.            |
| <b>ServletConfig</b> | <code>getServletConfig()</code>                               | Returns a <code>ServletConfig</code> object, which contains initialization and startup parameters for this servlet. |
| <b>String</b>        | <code>getServletInfo()</code>                                 | Returns information about the servlet, such as author, version, and copyright.                                      |
| void                 | <code>init(ServletConfig config)</code>                       | Called by the servlet container to indicate to a servlet that the servlet is being placed into service.             |
| void                 | <code>service(ServletRequest req, ServletResponse res)</code> | Called by the servlet container to allow the servlet to respond to a request.                                       |



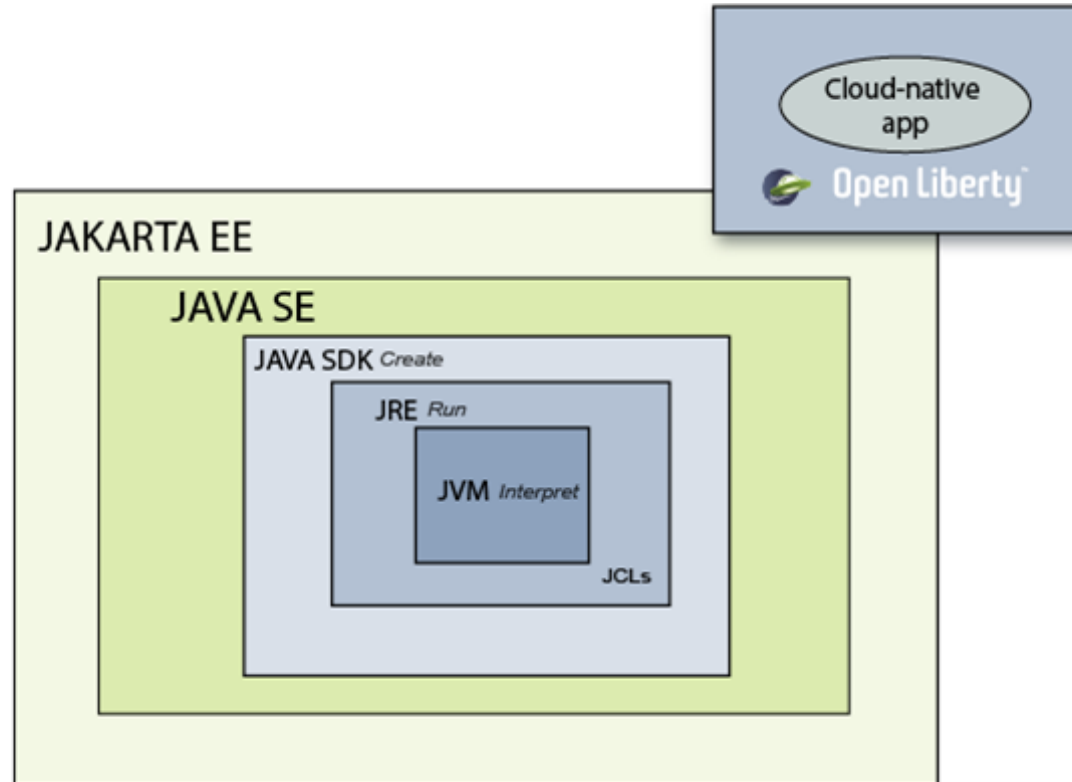
# Servlet Life Cycle



<http://volkanozturk.net/05-servlet-lifecycle/>



# Jakarta EE





# Deployment Descriptor

- web.xml is called “Deployment Descriptor” file
- A web application’s deployment descriptor describes the classes, resources and configuration of the application and how the web server uses them to serve web requests.
- It defines mappings between URL paths and the servlets that handle requests with those paths



# web.xml

```
<web-app>

<servlet>
  <servlet-name> </servlet-name>
  <servlet-class> </servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name> </servlet-name>
  <url-pattern> </url-pattern>
</servlet-mapping>

</web-app>
```



# Annotation

- Annotations start with '@'.
- Annotations do not change the action of a compiled program.
- Annotations help to associate metadata (information) to the program elements i.e. instance variables, constructors, methods, classes, etc.
- Annotations are not pure comments as they can change the way a program is treated by the compiler. See below code for example.
- Annotations basically are used to provide additional information, so could be an alternative to XML and Java marker interfaces.





# ServletConfig

| Modifier and Type                      | Method                                      | Description                                                                                                                                                                                                       |
|----------------------------------------|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>String</code>                    | <code>getInitParameter (String name)</code> | Gets the value of the initialization parameter with the given name.                                                                                                                                               |
| <code>Enumeration&lt;String&gt;</code> | <code>getInitParameterNames ()</code>       | Returns the names of the servlet's initialization parameters as an <code>Enumeration</code> of <code>String</code> objects, or an empty <code>Enumeration</code> if the servlet has no initialization parameters. |
| <code>ServletContext</code>            | <code>getServletContext ()</code>           | Returns a reference to the <code>ServletContext</code> in which the caller is executing.                                                                                                                          |
| <code>String</code>                    | <code>getServletName ()</code>              | Returns the name of this servlet instance.                                                                                                                                                                        |



# ServletRequest

- Defines an object to provide client request information to a servlet. The servlet container creates a `ServletRequest` object and passes it as an argument to the servlet's service method.
- A `ServletRequest` object provides data including parameter name and values, attributes, and an input stream. Interfaces that extend `ServletRequest` can provide additional protocol-specific data (for example, HTTP data is provided by [HttpServletRequest](#)).

# ServletRequest

| Modifier and Type        | Method                                          | Description                                                                                                               |
|--------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>String</b>            | <code>getParameter (String name)</code>         | Returns the value of a request parameter as a <code>String</code> , or <code>null</code> if the parameter does not exist. |
| <b>RequestDispatcher</b> | <code>getRequestDispatcher (String path)</code> | Returns a <code>RequestDispatcher</code> object that acts as a wrapper for the resource located at the given path.        |
| <b>String</b>            | <code>getServerName ()</code>                   | Returns the host name of the server to which the request was sent.                                                        |
| <b>int</b>               | <code>getServerPort ()</code>                   | Returns the port number to which the request was sent.                                                                    |
| <b>ServletContext</b>    | <code>getServletContext ()</code>               | Gets the servlet context to which this <code>ServletRequest</code> was last dispatched.                                   |
| <b>ServletContext</b>    | <code>getServletContext ()</code>               | Gets the servlet context to which this <code>ServletRequest</code> was last dispatched.                                   |



# ServletResponse

- Defines an object to assist a servlet in sending a response to the client. The servlet container creates a `ServletResponse` object and passes it as an argument to the servlet's service method.
- To send binary data in a MIME body response, use the `ServletOutputStream` returned by `getOutputStream()`. To send character data, use the `PrintWriter` object returned by `getWriter()`. To mix binary and text data, for example, to create a multipart response, use a `ServletOutputStream` and manage the character sections manually.

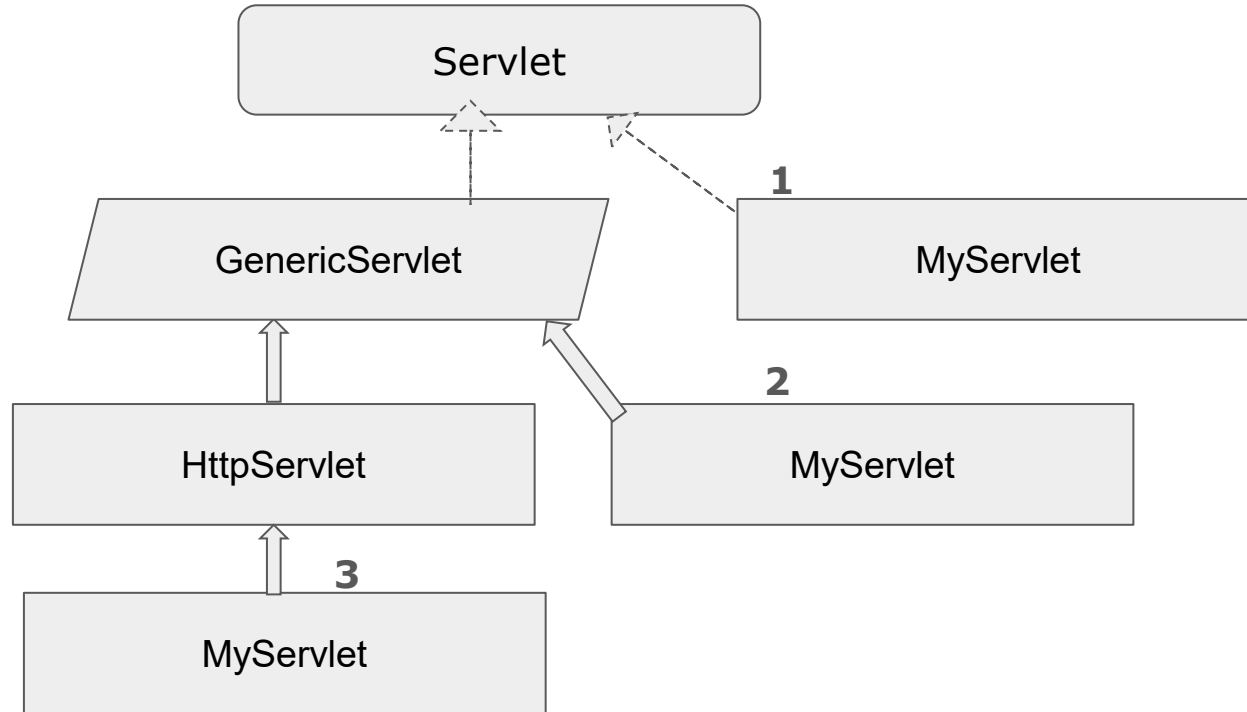


# ServletResponse

| Modifier and Type                | Method                                   | Description                                                                                                 |
|----------------------------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>ServletOutputStream</code> | <code>getOutputStream()</code>           | Returns a <code>ServletOutputStream</code> suitable for writing binary data in the response.                |
| <code>PrintWriter</code>         | <code>getWriter()</code>                 | Returns a <code>PrintWriter</code> object that can send character text to the client.                       |
| <code>void</code>                | <code>setContentType(String type)</code> | Sets the content type of the response being sent to the client, if the response has not been committed yet. |



# Three Ways to Create Servlet





# Create a Directory Structure



# Practical Time: Create a Servlet





# GenericServlet

- Defines a generic, protocol-independent servlet. To write an HTTP servlet for use on the Web, extend [HttpServlet](#) instead.
- GenericServlet implements the Servlet and ServletConfig interfaces. GenericServlet may be directly extended by a servlet, although it's more common to extend a protocol-specific subclass such as HttpServlet.
- GenericServlet makes writing servlets easier. It provides simple versions of the lifecycle methods init and destroy and of the methods in the ServletConfig interface. GenericServlet also implements the log method, declared in the ServletContext interface.
- To write a generic servlet, you need only override the abstract service method.



# GenericServlet

| Modifier and Type     | Method                               | Description                                                                                                           |
|-----------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| void                  | <b>destroy()</b>                     | Called by the servlet container to indicate to a servlet that the servlet is being taken out of service.              |
| <b>String</b>         | <b>getInitParameter(String name)</b> | Returns a String containing the value of the named initialization parameter, or null if the parameter does not exist. |
| <b>ServletConfig</b>  | <b>getServletConfig()</b>            | Returns this servlet's <b>ServletConfig</b> object.                                                                   |
| <b>ServletContext</b> | <b>getServletContext()</b>           | Returns a reference to the <b>ServletContext</b> in which this servlet is running.                                    |
| <b>String</b>         | <b>getServletInfo()</b>              | Returns information about the servlet, such as author, version, and copyright.                                        |
| <b>String</b>         | <b>getServletName()</b>              | Returns the name of this servlet instance.                                                                            |



# GenericServlet

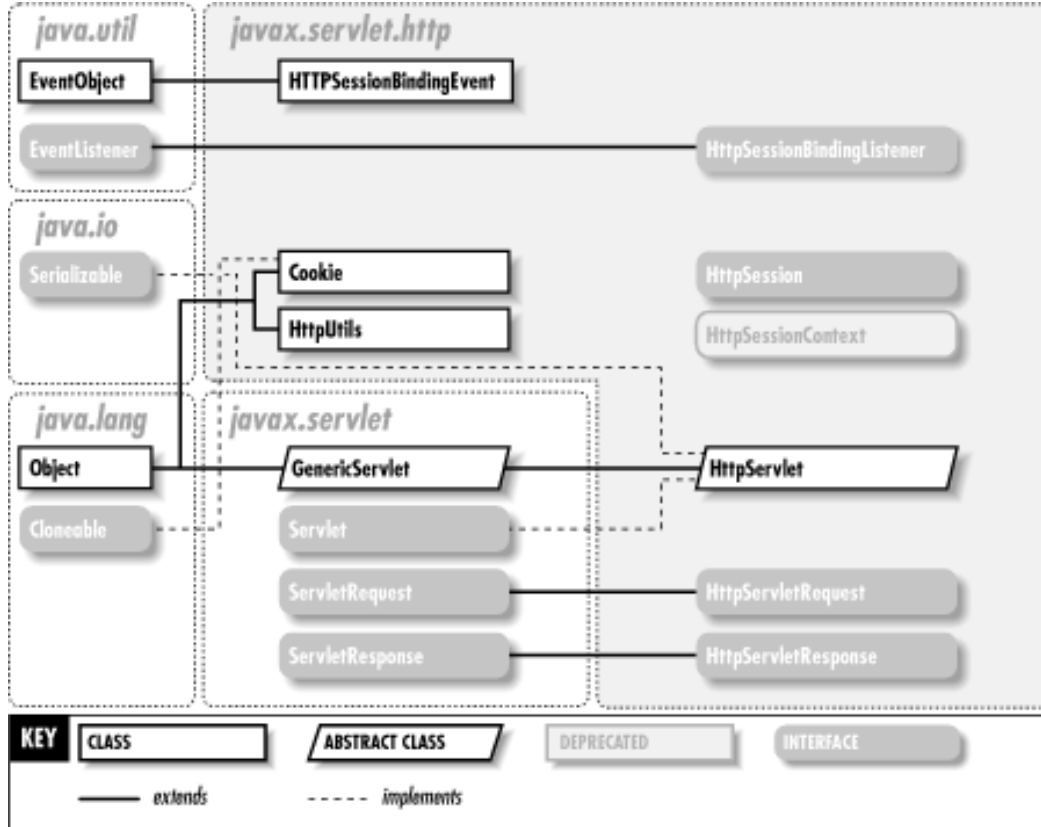
| Type          | Method                                                  | Description                                                                                                    |
|---------------|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| void          | <b>init()</b>                                           | A convenience method which can be overridden so that there's no need to call <code>super.init(config)</code> . |
| void          | <b>init(ServletConfig config)</b>                       | Called by the servlet container to indicate to a servlet that the servlet is being placed into service.        |
| abstract void | <b>service(ServletRequest req, ServletResponse res)</b> | Called by the servlet container to allow the servlet to respond to a request.                                  |



# Practical Time: Create a Generic Servlet



# jakarta.servlet.http





# HttpServlet

Provides an abstract class to be subclassed to create an HTTP servlet suitable for a Web site. A subclass of HttpServlet must override at least one method, usually one of these:

- doGet, if the servlet supports HTTP GET requests
- doPost, for HTTP POST requests
- doPut, for HTTP PUT requests
- delete, for HTTP DELETE requests
- init and destroy, to manage resources that are held for the life of the servlet
- getServletInfo, which the servlet uses to provide information about itself

There's almost no reason to override the service method, doOptions and doTrace methods.

# HttpServlet

| Modifier and Type | Method and Description                                                                                                                                                                           |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| protected void    | <b>doDelete</b> ( <b>HttpServletRequest</b> req, <b>HttpServletResponse</b> resp)<br>Called by the server (via the <code>service</code> method) to allow a servlet to handle a DELETE request.   |
| protected void    | <b>doGet</b> ( <b>HttpServletRequest</b> req, <b>HttpServletResponse</b> resp)<br>Called by the server (via the <code>service</code> method) to allow a servlet to handle a GET request.         |
| protected void    | <b>doHead</b> ( <b>HttpServletRequest</b> req, <b>HttpServletResponse</b> resp)<br>Receives an HTTP HEAD request from the protected <code>service</code> method and handles the request.         |
| protected void    | <b>doOptions</b> ( <b>HttpServletRequest</b> req, <b>HttpServletResponse</b> resp)<br>Called by the server (via the <code>service</code> method) to allow a servlet to handle a OPTIONS request. |

# HttpServlet

| Modifier and Type | Method and Description                                                                                                                                                                                                                       |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| protected<br>void | <b>doPost</b> ( <b>HttpServletRequest</b> req, <b>HttpServletResponse</b> resp)<br>Called by the server (via the <code>service</code> method) to allow a servlet to handle a POST request.                                                   |
| protected<br>void | <b>doPut</b> ( <b>HttpServletRequest</b> req, <b>HttpServletResponse</b> resp)<br>Called by the server (via the <code>service</code> method) to allow a servlet to handle a PUT request.                                                     |
| protected<br>void | <b>doTrace</b> ( <b>HttpServletRequest</b> req, <b>HttpServletResponse</b> resp)<br>Called by the server (via the <code>service</code> method) to allow a servlet to handle a TRACE request.                                                 |
| protected<br>void | <b>service</b> ( <b>HttpServletRequest</b> req, <b>HttpServletResponse</b> resp)<br>Receives standard HTTP requests from the public <code>service</code> method and dispatches them to the <code>doXXX</code> methods defined in this class. |
| void              | <b>service</b> ( <b>ServletRequest</b> req, <b>ServletResponse</b> res)<br>Dispatches client requests to the protected <code>service</code> method.                                                                                          |



➤➤ **Thank You..**